TIP

To check that an answer with eight bits is sensible, remember that the maximum denary value possible in seven bits is $2^7 - 1$ which is 127 whereas eight bits can hold values up to $2^8 - 1$ which is 255.

Conversions for hexadecimal numbers

It is possible to convert a hexadecimal number to denary by using the method shown in Table 1.03. However, if there are more than a few digits, the numbers involved in the conversion become very large. Instead, the sensible approach is to first convert the hexadecimal number to a binary number which can then be converted to denary.

To convert a hexadecimal number to binary, each digit is treated separately and converted into a 4-bit binary equivalent, remembering that F converts to 1111, E converts to 1110 and so on.

To convert a binary number to hexadecimal you start with the four least significant bits and convert them to one hexadecimal digit. You then proceed upwards towards the most significant bit, successively taking groupings of four bits and converting each grouping to the corresponding hexadecimal digit.

TASK 1.01

Convert each of the denary numbers 96, 215 and 374 into hexadecimal numbers.
Convert each of the hexadecimal numbers B4, FF and 3A2C to denary numbers.

Question 1.01

Does a computer ever use hexadecimal numbers?

1.02 Numbers and quantities

There are several different types of numbers within the denary system. Examples of these are provided in Table 1.05.

Type of number	Examples	Comments
Integer	3 or 47	A whole number used for counting
Signed integer	−3 or 47	The positive number has an implied + sign
Fraction	2/3 or 52/17	Rarely used in computer science
A number with a whole number part and a fractional number part	−37.85 or 2.83	The positive number has an implied + sign
A number expressed in exponential notation	-3.6×10^8 or 4.2×10^{-9}	The value can be positive or negative and the exponent can be positive or negative

Table 1.05 Different ways to express a value using the denary number system

We will focus on how large values are represented. If we have a quantity that includes units of measurement, it can be written in three different ways. For example, a distance could be written in any one of these three ways:

- 23 567 m
- 23.567×10^3 m
- 23.567 km

The second example has used an exponential notation to define the magnitude of the value. The third example has added a prefix to the unit to define this magnitude. We read this as 23.567 kilometres.

The 'kilo' is an example of a **decimal prefix**. There are four decimal prefixes commonly used for large numbers. These are shown in Table 1.06.

Decimal prefix name	Symbol used	Factor applied to the value
kilo	k	10^3
mega	M	10^6
giga	G	10^9
tera	T	10^{12}

Table 1.06 The decimal prefixes

KEY TERM

Decimal prefix: examples are kilo, mega, giga and tera representing factors of 10^3, 10^6, 10^9 and 10^{12} respectively to define the magnitude of a value.

Unfortunately, for a long time the computing world used these prefix names but with a slightly different definition. The value for 2^{10} is 1024. Because this is close to 1000, computer scientists decided that they could use the kilo prefix to represent 1024. So, for example, if a computer system had the following values quoted for the processor speed and the size of the memory and of the hard disk:

Processor speed	1.6 GHz
Size of RAM	8 GB
Size of hard disk	400 GB

The prefix G would represent 10^9 for the processor speed but would almost certainly represent 1024 × 1024 × 1024 for the other two values.

This unsatisfactory situation has now been resolved by the definition of a new set of names which can be used to define a **binary prefix**. A selection of these is shown in Table 1.07.

Binary prefix name	Symbol used	Factor applied to the value
kibi	Ki	2^{10}
mebi	Mi	2^{20}
gibi	Gi	2^{30}
tebi	Ti	2^{40}

Table 1.07 Some examples of binary prefixes

KEY TERM

Binary prefix: examples are kibi, mebi, gibi and tebi representing factors of 2^{10}, 2^{20}, 2^{30} and 2^{40} respectively to define the magnitude of a value.

When a number or a quantity is presented for a person to read it is best presented with either one denary digit or two denary digits before the decimal point. If a calculation has been carried out, the initial result found may not match this requirement. A conversion of

the presented value will be needed by choosing a sensible magnitude factor. For example, consider the following two answers calculated for the size of a file:

a 34 560 bytes

Here, a conversion to kibibytes would be sensible using the calculation:

$$344560\text{B} = \frac{34560}{1024}\text{KiB} = 33.75\,\text{KiB}$$

b 3 456 000 bytes

Here, a conversion to mebibytes would be sensible using the calculation:

$$3456000\text{B} = \frac{\left(\frac{3456000}{1024}\right)}{1024}\text{MiB} = 3.296\,\text{MiB}$$

If a calculation is to be performed with values quoted with different magnitude factors there must first be conversions to ensure all values have the same magnitude factor. For example, if you needed to know how many files of size 2.4 MiB could be stored on a 4 GiB memory stick there should be a conversion of the GiB value to the corresponding MiB value.

The calculation would be:

$$\frac{(4 \times 1024)\text{MiB}}{2.4\text{MiB}} = 1076$$

1.03 Internal coding of numbers

The discussion in this chapter relates only to the coding of integer values. The coding of non-integer numeric values (real numbers) is considered in Chapter 16 (Section 16.03).

Coding for integers

Computers need to store integer values for a number of purposes. Sometimes only a simple integer is stored, with the understanding that it is a positive number. This is stored simply as a binary number. The only decision to be made is how many bytes should be used. If the choice is to use two bytes (16 bits) then the range of values that can be represented is 0 to $(2^{16} - 1)$ which is 0 to 65 535.

However, in many cases we need to identify whether the number is positive or negative, so we use a signed integer. A signed integer can just have the binary code for the value with an extra bit to define the sign. This is referred to as 'sign and magnitude representation'. For this the convention is to use a 0 to represent + and a 1 to represent –. A few examples of this are shown in Table 1.08.

However, there are a number of disadvantages to using this format, so signed integers are usually in **two's complement** form. Here we need two definitions.

The **one's complement** of a binary number is defined as the binary number obtained if each binary digit is individually subtracted from 1. This means that each 0 is switched to 1 and each 1 switched to 0. The two's complement is defined as the binary number obtained if 1 is added to the one's complement number.

KEY TERMS

One's complement: the binary number obtained by subtracting each digit in a binary number from 1

Two's complement: the one's complement of a binary number, plus 1